Experiment-3

Python code

In [2]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

# STEP 1: Load the image
image = cv2.imread("image.jpg.png")

# Check if image is loaded
if image is None:
    print("Error: Image not found!")
    exit()

# Convert BGR image to RGB for Matplotlib
original = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# STEP 2: Apply Gaussian Filter
gaussian = cv2.GaussianBlur(image, (5, 5), 0)

# STEP 3: Apply Median Filter
median = cv2.medianBlur(image, 5)

# STEP 4: Apply Average (Mean) Filter
average_kernel = np.ones((5, 5), np.float32) / 25
average = cv2.filter2D(image, -1, average_kernel)

# STEP 5: Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# STEP 6: Apply Laplacian Filter
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

# STEP 7: Apply Sobel Edge Detection
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

# Combine Sobel X and Y
sobel_combined = cv2.addWeighted(
    sobel_x, 0.5,
    sobel_y, 0.5,
    0
)

# Convert filtered images to RGB
gaussian_rgb = cv2.cvtColor(gaussian, cv2.COLOR_BGR2RGB)
median_rgb = cv2.cvtColor(median, cv2.COLOR_BGR2RGB)
average_rgb = cv2.cvtColor(average, cv2.COLOR_BGR2RGB)
```

```
# STEP 8: Display all images
plt.figure(figsize=(15, 10))

plt.subplot(2, 4, 1)
plt.imshow(original)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 4, 2)
plt.imshow(gaussian_rgb)
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(median_rgb)
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(average_rgb)
plt.title("Average Filter")
plt.axis("off")

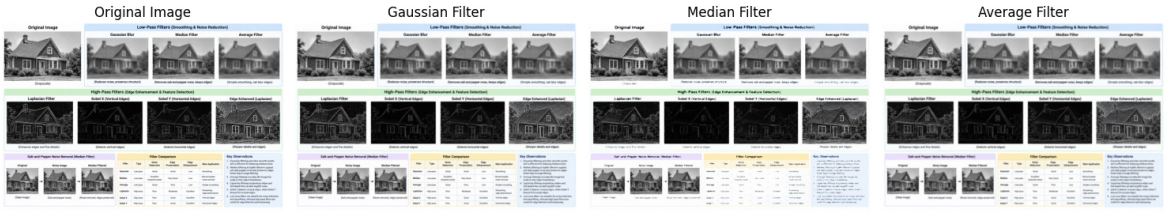
plt.subplot(2, 4, 5)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

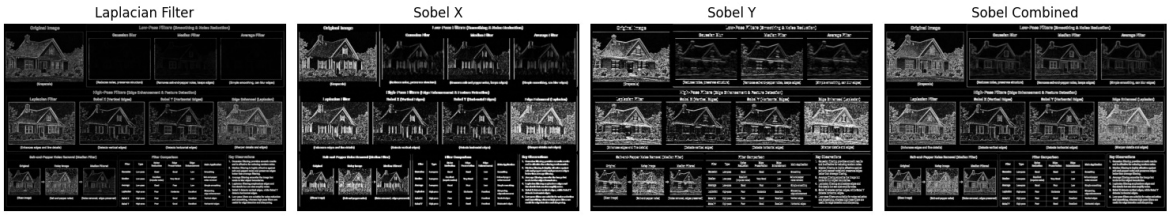
plt.subplot(2, 4, 6)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(2, 4, 8)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Sobel Combined")
plt.axis("off")

plt.tight_layout()
plt.show()
```





In [4]:
```python
#step 9 Compare computional performance
start = time.time()
cv2.blur(image, (5, 5))
average_time = time.time() - start

start = time.time()
cv2.GaussianBlur(image, (5, 5), 0)
gaussian_time = time.time() - start

start = time.time()
cv2.medianBlur(image, 5)
median_time = time.time() - start

start = time.time()
cv2.Laplacian(gray, cv2.CV_64F)
laplacian_time = time.time() - start

start = time.time()
cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_time = time.time() - start


print("\n--- Computational Performance ---")
print("Average Filter Time:", average_time, "seconds")
print("Gaussian Filter Time:", gaussian_time, "seconds")
print("Median Filter Time:", median_time, "seconds")
print("Laplacian Filter Time:", laplacian_time, "seconds")
print("Sobel Filter Time:", sobel_time, "seconds")

#step 10 Display observations and interpretations

print("\n--- Observations ---")

print("1. Average Filter:")
print("   Reduces noise but may blur edges.")

print("\n2. Gaussian Filter:")
print("   Provides smooth results and preserves structures better.")

print("\n3. Median Filter:")
print("   Best for removing salt-and-pepper noise.")

print("\n4. Laplacian Filter:")
```

```
print("   Detects edges and enhances fine details.")

print("\n5. Sobel Filter:")
print("   Detects horizontal and vertical edges.")

print("\n--- Conclusion ---")
print("Low-pass filters are useful for image smoothing and noise reduction.")
print("High-pass filters are useful for edge detection and image sharpening.")
print("The choice of filter depends on the application and image type.")
```

```
--- Computational Performance ---
Average Filter Time: 0.013513326644897461 seconds
Gaussian Filter Time: 0.0019910335540771484 seconds
Median Filter Time: 0.013001203536987305 seconds
Laplacian Filter Time: 0.01201319694519043 seconds
Sobel Filter Time: 0.01212000846862793 seconds

--- Observations ---
1. Average Filter:
   Reduces noise but may blur edges.

2. Gaussian Filter:
   Provides smooth results and preserves structures better.

3. Median Filter:
   Best for removing salt-and-pepper noise.

4. Laplacian Filter:
   Detects edges and enhances fine details.

5. Sobel Filter:
   Detects horizontal and vertical edges.

--- Conclusion ---
Low-pass filters are useful for image smoothing and noise reduction.
High-pass filters are useful for edge detection and image sharpening.
The choice of filter depends on the application and image type.
```